Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

***תזכורת** : "מתכנת ג'וניור מוכשר" ושחובה לבדוק אותו.

שאלה : למה מודל שכותבת קוד סביר ברוב המקרים - ממציאה פונקציה שלא קיימת ״בבטחון״? **מה דעתכם?**

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

***תזכורת** : "מתכנת ג'וניור מוכשר" ושחובה לבדוק אותו.

שאלה : למה מודל שכותבת קוד סביר ברוב המקרים - ממציאה פונקציה שלא קיימת ״בבטחון״?

1. באג?
2. המודל עוד לא מספיק טוב?

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

***תזכורת** : "מתכנת ג'וניור מוכשר" ושחובה לבדוק אותו.

שאלה : למה מודל שכותבת קוד סביר ברוב המקרים - ממציאה פונקציה שלא קיימת ״בבטחון״?

1. ~~באג~~?
2. ~~המודל עוד לא מספיק טוב~~?

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

הסיבה: אופן שבו המודל בנוי: **ממה הכל מתחיל?**

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

הסיבה: אופן שבו המודל בנוי: **מה הלאה?**

טקסט

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

הסיבה: אופן שבו המודל בנוי: **מה הלאה?**

טקסט => טוקנים

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

הסיבה: אופן שבו המודל בנוי: **מה הלאה?**

טקסט => טוקנים => embeddings

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

הסיבה: אופן שבו המודל בנוי: **מה הלאה?**

טקסט => טוקנים => embeddings <= Transformer

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

הסיבה: אופן שבו המודל בנוי: **מה הלאה?**

טקסט => טוקנים => embeddings => Transformer => (חוזרים) טוקן חדש

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

הסיבה: אופן שבו המודל בנוי: **מה בסוף?**

טקסט => טוקנים => embeddings => Transformer => (חוזרים) טוקן חדש

=> טמפרטורה

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

הסיבה: אופן שבו המודל בנוי: **מה בסוף?**

טקסט => טוקנים => embeddings => Transformer => (חוזרים) טוקן חדש

=> טמפרטורה => הסתברות על תוקן הבא.

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

הסיבה: אופן שבו המודל בנוי: **ממה הכל מתחיל?**

טקסט => טוקנים => embeddings => Transformer => (חוזרים) טוקן חדש

=> טמפרטורה => הסתברות על תוקן הבא.

שאלה : איפה לדעתכם הבעיה נמצאת?

Vibe Coding

שיעור 2

במפגש הקודם:

ראינו הזכרנו מודל שכתב בביטחון מלא

```
df.to_excel_fast("out.xlsx")    # ← פונקציה בדויה !
```

הסיבה: אופן שבו המודל בנוי: **ממה הכל מתחיל?**

טקסט => טוקנים => embeddings => Transformer => (חוזרים) טוקן חדש

=> טמפרטורה => הסתברות על תוקן הבא.

שאלה : איפה לדעתכם הבעיה נמצאת?

תשובה : בתוך חלון ההקשר.

ד״ר ולד לנדא

Vibe Coding

שיעור 2

מטרות המפגש:

1. להסביר איך טקסט וקוד הופכים לטוקנים, ולהעריך כמה טוקנים "עולה" טקסט בעברית לעומת אנגלית.

ד״ר ולד לנדא

Vibe Coding

שיעור 2

מטרות המפגש:

1. להסביר איך טקסט וקוד הופכים לטוקנים, ולהעריך כמה טוקנים "עולה" טקסט בעברית לעומת אנגלית.
2. להסביר מהם embeddings ולמה "קרבה במרחב" מייצגת קרבה במשמעות.

ד״ר ולד לנדא

Vibe Coding

שיעור 2

מטרות המפגש:

1. להסביר איך טקסט וקוד הופכים לטוקנים, ולהעריך כמה טוקנים "עולה" טקסט בעברית לעומת אנגלית.
2. להסביר מהם embeddings ולמה "קרבה במרחב" מייצגת קרבה במשמעות.
3. לתאר את הרעיון של Attention ואת תפקידו בארכיטקטורת ה-Transformer - ברמת אינטואיציה.

ד״ר ולד לנדא

Vibe Coding

שיעור 2

מטרות המפגש:

1. להסביר איך טקסט וקוד הופכים לטוקנים, ולהעריך כמה טוקנים "עולה" טקסט בעברית לעומת אנגלית.
2. להסביר מהם embeddings ולמה "קרבה במרחב" מייצגת קרבה במשמעות.
3. לתאר את הרעיון של Attention ואת תפקידו בארכיטקטורת ה-Transformer - ברמת אינטואיציה.
4. להסביר איך המודל בוחר את הטוקן הבא ומה בדיוק עושה פרמטר הטמפרטורה (כולל חישוב מספרי).

ד״ר ולד לנדא

Vibe Coding

שיעור 2

מטרות המפגש:

1. להסביר איך טקסט וקוד הופכים לטוקנים, ולהעריך כמה טוקנים "עולה" טקסט בעברית לעומת אנגלית.
2. להסביר מהם embeddings ולמה "קרבה במרחב" מייצגת קרבה במשמעות.
3. לתאר את הרעיון של Attention ואת תפקידו בארכיטקטורת ה-Transformer - ברמת אינטואיציה.
4. להסביר איך המודל בוחר את הטוקן הבא ומה בדיוק עושה פרמטר הטמפרטורה (כולל חישוב מספרי).
5. להסביר מהו חלון הקשר, מה קורה כשהוא מתמלא, ואיך זה משפיע על איכות התשובות.

ד״ר ולד לנדא

Vibe Coding

שיעור 2

מטרות המפגש:

1. להסביר איך טקסט וקוד הופכים לטוקנים, ולהעריך כמה טוקנים "עולה" טקסט בעברית לעומת אנגלית.
2. להסביר מהם embeddings ולמה "קרבה במרחב" מייצגת קרבה במשמעות.
3. לתאר את הרעיון של Attention ואת תפקידו בארכיטקטורת ה-Transformer - ברמת אינטואיציה.
4. להסביר איך המודל בוחר את הטוקן הבא ומה בדיוק עושה פרמטר הטמפרטורה (כולל חישוב מספרי).
5. להסביר מהו חלון הקשר, מה קורה כשהוא מתמלא, ואיך זה משפיע על איכות התשובות.
6. להסביר מנגנונית למה הזיות קורות, לזהות חמישה סוגי הזיות בקוד, ולאמת כל אחד מהם.

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

חשיבות: כמה "זיכרון" יש למודל, כמה עולה השימוש בו, הבדלים בין עברית לאנגלית (ושפות אחרות), הכל נמדד ביחידה אחת: **טוקן**.

מה זה טוקן? למה צריך כזה?

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

חשיבות: כמה "זיכרון" יש למודל, כמה עולה השימוש בו, הבדלים בין עברית לאנגלית (ושפות אחרות), הכל נמדד ביחידה אחת: **טוקן**.

הסבר: רשת עצבית יודעת לעבוד רק עם מספרים. לכן השלב הראשון הוא טוקניזציה (**tokenization**): פירוק הטקסט לחתיכות, ומיפוי של כל חתיכה למספר מזהה מתוך "אוצר מילים" קבוע (vocabulary) של המודל (לרוב כמה עשרות עד כמה מאות אלפי טוקנים).

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

טוקניזציה (**tokenization**): פירוק הטקסט לחתיכות, ומיפוי של כל חתיכה למספר מזהה מתוך "אוצר מילים" קבוע (vocabulary) של המודל.

טוקן הוא לא בהכרח מילה. הוא יכול להיות:

- מילה שלמה ונפוצה (**average**).

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

טוקניזציה (**tokenization**): פירוק הטקסט לחתיכות, ומיפוי של כל חתיכה למספר מזהה מתוך "אוצר מילים" קבוע (vocabulary) של המודל.

טוקן הוא לא בהכרח מילה. הוא יכול להיות:

- מילה שלמה ונפוצה (**average**).
- חלק ממילה (**program + ming**).

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

טוקניזציה (**tokenization**): פירוק הטקסט לחתיכות, ומיפוי של כל חתיכה למספר מזהה מתוך "אוצר מילים" קבוע (vocabulary) של המודל.

טוקן הוא לא בהכרח מילה. הוא יכול להיות:

- מילה שלמה ונפוצה (**average**).
- חלק ממילה (**program + ming**).
- סימן בודד או צירוף סימנים (**):**).

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

טוקניזציה (**tokenization**): פירוק הטקסט לחתיכות, ומיפוי של כל חתיכה למספר מזהה מתוך "אוצר מילים" קבוע (vocabulary) של המודל.

טוקן הוא לא בהכרח מילה. הוא יכול להיות:

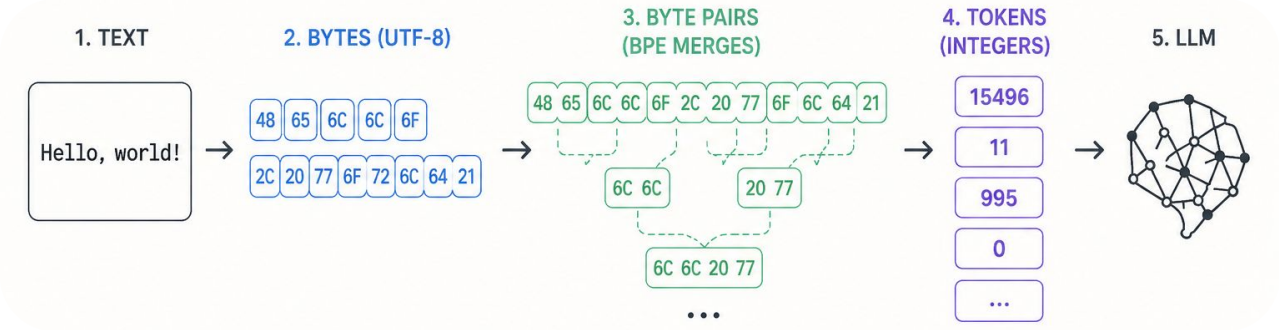
- מילה שלמה ונפוצה (**average**).
- חלק ממילה (**program + ming**).
- סימן בודד או צירוף סימנים (**):**).
- ובמקרים קיצוניים - אפילו חלק מאות, ברמת הבייטים של הקידוד.

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

איך נקבע אוצר המילים? בשלב הכנה לפני אימון המודל, אלגוריתם (בדרך כלל ממשפחת BPE - Byte Pair Encoding).

- סורק כמות עצומה של טקסט ומאחד שוב ושוב את צמדי החתיכות השכיחים ביותר ליחידה אחת.
- **רצפים שכיחים בנתוני האימון הופכים לטוקן אחד, ורצפים נדירים מתפרקים לחתיכות קטנות.**



ד״ר ולד לנדא

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

ניסוח פורמלי - טוקנייזר ואלגוריתם BPE:

הקורפוס: נניח שזה כל הטקסט שקיים בעולם: hug hug hug hugs bug bugs bug bugs hug hug hugs hugs

השכיחויות: מפרקים את הקורפוס למילים וסופרים כמה פעמים כל מילה w מופיעה - זו השכיחות $f(w)$:

$$f(\mathtt{hug}) = 5, \quad f(\mathtt{hugs}) = 3, \quad f(\mathtt{bug}) = 2, \quad f(\mathtt{bugs}) = 2$$

בניית אוצר המילים: מתחילים מהאותיות: $\mathcal{V} = \{\mathtt{b}, \mathtt{g}, \mathtt{h}, \mathtt{s}, \mathtt{u}\}$, וכל מילה היא רצף של אותיות (למשל $\mathtt{hugs} = \mathtt{h} \cdot \mathtt{u} \cdot \mathtt{g} \cdot \mathtt{s}$). בכל צעד:

1. **סופרים** כל זוג טוקנים צמודים, כשכל הופעה בתוך מילה w נספרת $f(w)$ פעמים. למשל, `u · g` מופיע **פעם אחת** בכל אחת מארבע המילים, ולכן $\mathrm{count}(\mathtt{u}, \mathtt{g}) = 5 * 1 + 3 * 1 + 2 * 1 + 2 * 1 = 12$
2. **ממזגים** את הזוג השכיח ביותר לטוקן חדש, מוסיפים אותו ל-$\mathcal{V}$, ומחליפים אותו בכל המילים.
3. **מתי עוצרים**? : **הגעה לגודל מילון היעד** או **סף תדירות מינימלי**.

צעד	הזוג השכיח ביותר (ספירה)	טוקן חדש ב-V
1	`u · g` (12)	`ug`
2	`h · ug` (8)	`hug`
3	`b · ug` (4)	`bug`
4	`hug · s` (3)	`hugs`

נוסחה: אם $c_w(a, b)$ הוא מספר הפעמים שהזוג (a, b) מופיע צמוד בתוך המילה w, אז

$$\mathrm{count}(a, b) = \sum_w f(w) \cdot c_w(a, b)$$

ובכל צעד ממזגים את $(a^*, b^*) = \arg\max_{(a,b)} \mathrm{count}(a, b)$ - **הזוג** עם הספירה הגבוהה ביותר.

ד״ר ולד לנדא

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

ניסוח פורמלי - טוקנייזר ואלגוריתם BPE:

התוצאה: אחרי 4 צעדים $\mathcal{V} = \{\mathrm{b}, \mathrm{g}, \mathrm{h}, \mathrm{s}, \mathrm{u}, \mathrm{ug}, \mathrm{hug}, \mathrm{bug}, \mathrm{hugs}\}$. הטוקנייזר $\mathcal{T}$ מפרק טקסט חדש בעזרת אותם מיזוגים, לפי הסדר שבו נלמדו, ומחזיר רצף טוקנים $\tau(\text{טסקט}) = (t_1, \ldots, t_n)$, כאשר n הוא מספר הטוקנים. למשל: $\tau(\mathrm{hugs}) = (\mathrm{hugs})$ עם n = 1, אבל $\tau(\mathrm{bugs}) = (\mathrm{bug}, \mathrm{s})$ עם n = 2 - כי `*bugs*` פחות שכיחה, והמיזוג `*bug · s*` לא נלמד. ומילה שאף זוג בה לא נלמד, כמו `*bush*`, נשארת ברמת האות: n = 4.

Iteration	*Corpus*	*Vocabulary*
0	A A C G C A C T A T A T A	{A,T,C,G}
1	A A C G C A C T A T A T A	{A,T,C,G,TA}
2	A A C G C A C TA TA TA	{A,T,C,G,TA, AC}
3	A AC G C AC TA TA TA	.....

Figure 2: Illustration of the BPE vocabulary constructions.

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

דוגמה:

מפענח o200k_base - משמש מודלים של OpenAI מ-2024.

```
"Hello world!"                    →  3 טוקנים:  Hello | world | !
"programming"                     →  2 טוקנים:  program | ming
"def average(numbers):"           →  4 טוקנים:  def | average | (numbers | ):
"calculate_total_price_with_vat"  →  6 טוקנים:  calculate | _total | _price | _with | _v | at
```

1. **הרווח שייך לטוקן.** `world` (עם רווח מוביל) הוא טוקן אחר מ-`world`.
2. **קוד מתפרק לפי מה שנפוץ בקוד**, לא לפי כללי התחביר. `numbers)` נדבק לסוגריים כי הצירוף הזה שכיח בקוד שהמודל ראה.
3. **שם משתנה ארוך נשבר לחלקים.** `vat_` נדיר יותר מ-`price_`, ולכן התפרק ל-`v` + `at_`.

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

דוגמה: עברית מול אנגלית

מפענחים: cl100k_base (דור 2023) ו-o200k_base (דור 2024).

טקסט	cl100k	o200k
`Write a function that computes the average of a list of numbers` (12 מילים)	12 טוקנים	12 טוקנים
כתוב פונקציה שמחשבת ממוצע של רשימת מספרים (7 מילים)	42 טוקנים	15 טוקנים
שלום עולם!	11 טוקנים	3 טוקנים
התכנות	טוקנים 7	3 טוקנים

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

דוגמה: עברית מול אנגלית

מפענחים: cl100k_base (דור 2023) ו-o200k_base (דור 2024).

המשפט העברי ב-**o200k**:

כתוב | פונ | ק | ציה | שמ | חש | בת | ממ | וצ | ע | של | רש | ימת | מספר | ים

במפענח הישן **cl100k** - כמעט אות-אות, ולעיתים אפילו פחות מאות.
חלק מהאותיות העבריות נשברו לשני טוקנים, כי המפענח עבד ברמת הבייטים של הקידוד (אות עברית תופסת 2 בייטים ב-UTF-8, בעוד אות אנגלית תופסת בייט אחד).

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

דוגמה: עברית מול אנגלית

- באנגלית, כמעט כל מילה היא טוקן אחד. בעברית, גם במפענח המודרני, מילה ממוצעת היא כ-2 טוקנים (15 טוקנים ל-7 מילים).
- המפענחים משתפרים משמעותית בעברית - אבל הפער לא נעלם.
- **למה**?

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

דוגמה: עברית מול אנגלית

- באנגלית, כמעט כל מילה היא טוקן אחד. בעברית, גם במפענח המודרני, מילה ממוצעת היא כ-2 טוקנים (15 טוקנים ל-7 מילים).
- המפענחים משתפרים משמעותית בעברית - אבל הפער לא נעלם.
- **למה**? כי הנתונים שעליהם נבנה אוצר המילים הם ברובם אנגלית. רצפים עבריים פשוט נדירים יותר, ולכן מתפרקים לחתיכות קטנות יותר.

ד״ר ולד לנדא

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

מה המשמעות המעשית? מה לומדים מהדוגמא?

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

מה המשמעות המעשית?

1. **"זיכרון"**: אותו חלון הקשר (חלק ה') מכיל פחות טקסט עברי מאנגלי.

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

מה המשמעות המעשית?

1. **"זיכרון"**: אותו חלון הקשר (חלק ה') מכיל פחות טקסט עברי מאנגלי.
2. **עלות**: בכלים שמתמחרים לפי טוקן, אותה בקשה בעברית יכולה לעלות יותר.

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

מה המשמעות המעשית?

1. "**זיכרון**": אותו חלון הקשר (חלק ה') מכיל פחות טקסט עברי מאנגלי.
2. **עלות**: בכלים שמתמחרים לפי טוקן, אותה בקשה בעברית יכולה לעלות יותר.
3. **איכות**: מילה שמפורקת לחתיכות רבות היא "פחות מוכרת" למודל, כי היא הופיעה פחות באימון. זו אחת הסיבות האפשריות (לא היחידה) להבדלי האיכות בין עברית לאנגלית שבדקתם במפגש 1.

ד״ר ולד לנדא

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

טקסט	cl100k	o200k
`Write a function that computes the average of a list of numbers` (מילים 12)	12 טוקנים	12 טוקנים
כתוב פונקציה שמחשבת ממוצע של רשימת מספרים (7 מילים)	42 טוקנים	15 טוקנים
שלום עולם!	11 טוקנים	3 טוקנים
התכנות	טוקנים 7	3 טוקנים

ניסוח פורמלי - יחס טוקנים-למילה וקיבולת:

עבור טקסט עם n טוקנים ו-n_{words} מילים, נגדיר את **יחס הטוקנים-למילה**:

$$\rho = \frac{n}{n_{\text{words}}}$$

מהמדידה למעלה (o200k), למשפט שבטבלה: $\rho_{\text{he}} = 15/7 \approx 2.14$ ו- $\rho_{\text{en}} = 12/12 = 1.0$

קיבולת: חלון ההקשר מכיל לכל היותר L טוקנים, וחלק מהם צריך להישאר פנוי לתשובה של המודל - נסמן את מספרם ב-n_{answer}. המקום שנשאר לטקסט שלכם הוא $L_{\text{free}} = L - n_{\text{answer}}$, ומספר המילים שנכנסות בו הוא בקירוב:

$$n_{\text{words}} \approx \frac{L_{\text{free}}}{\rho}$$

דוגמה: $L = 128{,}000$ ו- $n_{\text{answer}} = 8{,}000$ כלומר $L_{\text{free}} = 120{,}000$ באנגלית נכנסות כ- $120{,}000/1.0 = 120{,}000$ מילים; בעברית רק כ- $120{,}000/2.14 \approx 56{,}000$. באותה מידה, בכלי שמתמחר לפי טוקן, אותו תוכן בעברית עולה פי $\rho_{\text{he}}/\rho_{\text{en}} \approx 2.1$.

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

מה המשמעות המעשית?

1. **"זיכרון"**: אותו חלון הקשר (חלק ה') מכיל פחות טקסט עברי מאנגלי.
2. **עלות**: בכלים שמתמחרים לפי טוקן, אותה בקשה בעברית יכולה לעלות יותר.
3. **איכות**: מילה שמפורקת לחתיכות רבות היא "פחות מוכרת" למודל, כי היא הופיעה פחות באימון. זו אחת הסיבות האפשריות (לא היחידה) להבדלי האיכות בין עברית לאנגלית שבדקתם במפגש 1.

טעות נפוצה : "טוקן = מילה." לא. לכן הערכות כמו "כתבתי 500 מילים, זה בטח 500 טוקנים" שגויות - בעברית זה יכול להיות פי שניים ויותר.

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

עקרונות מעשיים:

1. **קוד, שמות משתנים והערות בקוד - באנגלית**. זה חוסך טוקנים, וגם תואם את מה שהמודל ראה הכי הרבה באימון.

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

עקרונות מעשיים:

1. **קוד, שמות משתנים והערות בקוד - באנגלית**. זה חוסך טוקנים, וגם תואם את מה שהמודל ראה הכי הרבה באימון.
2. **ההסבר הטבעי לבקשה יכול להיות בעברית** - זה בסדר גמור, אבל כשהבקשה ארוכה ומורכבת, שווה לבדוק (כמו בתרגול מפגש 1) אם ניסוח באנגלית נותן תוצאה טובה יותר.

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

עקרונות מעשיים:

1. **קוד, שמות משתנים והערות בקוד - באנגלית**. זה חוסך טוקנים, וגם תואם את מה שהמודל ראה הכי הרבה באימון.
2. **ההסבר הטבעי לבקשה יכול להיות בעברית** - זה בסדר גמור, אבל כשהבקשה ארוכה ומורכבת, שווה לבדוק (כמו בתרגול מפגש 1) אם ניסוח באנגלית נותן תוצאה טובה יותר.
3. כשצריך להעריך **"כמה נכנס לחלון (context)"** - מדדו עם כלי ספירת טוקנים, אל תנחשו לפי מילים.

ד״ר ולד לנדא

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

שאלת בדיקה?:

למה המילה *programming* היא 2 טוקנים והמילה *התכנות* היא 3 טוקנים (ב-o200k), למרות שהמילה העברית קצרה יותר באותיות?

Vibe Coding

שיעור 2 - חלק א׳ : **טוקניזציה**

שאלת בדיקה?:

למה המילה *programming* היא 2 טוקנים והמילה *התכנות* היא 3 טוקנים (ב-o200k), למרות שהמילה העברית קצרה יותר באותיות?

תשובה: כי אורך הטוקניזציה נקבע לפי ***שכיחות*** הרצף בנתוני האימון, לא לפי אורכו. **<u>רצפים עבריים נדירים יותר</u>** בנתונים, ולכן מתפרקים ליותר חתיכות.

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

Embeddings: משמעות כמיקום במרחב.

חשיבות: אחרי הטוקניזציה יש לנו רשימת מספרים מזהים.

- אבל מספר מזהה לא אומר כלום על משמעות.
- הטוקן ***dog*** יכול לקבל את המספר 5,000 והטוקן ***cat*** את המספר 91,000.
- אין שום דבר במספרים שאומר שהם קרובים.
- הפתרון **Embeddings** - וגם מקור היכולת של המודל "להכליל", וגם מקור חלק מההזיות.

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

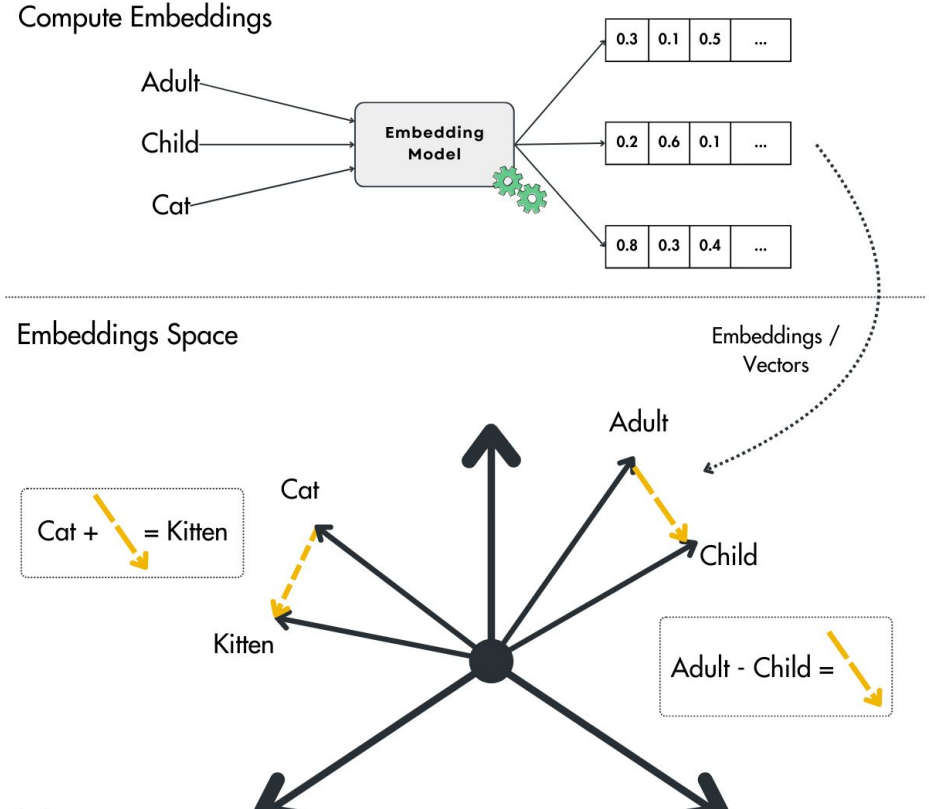
Embeddings: משמעות כמיקום במרחב.

הסבר: כל טוקן ממופה ל**וקטור** - רשימה ארוכה של מספרים (בדרך כלל אלפי מספרים).

- חשובים על הווקטור כעל **נקודה במרחב רב-ממדי**.
- המיקומים האלה אינם נקבעים ידנית: הם נלמדים במהלך האימון, כך שטוקנים שמופיעים **בהקשרים דומים** מקבלים מיקומים **קרובים**.

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

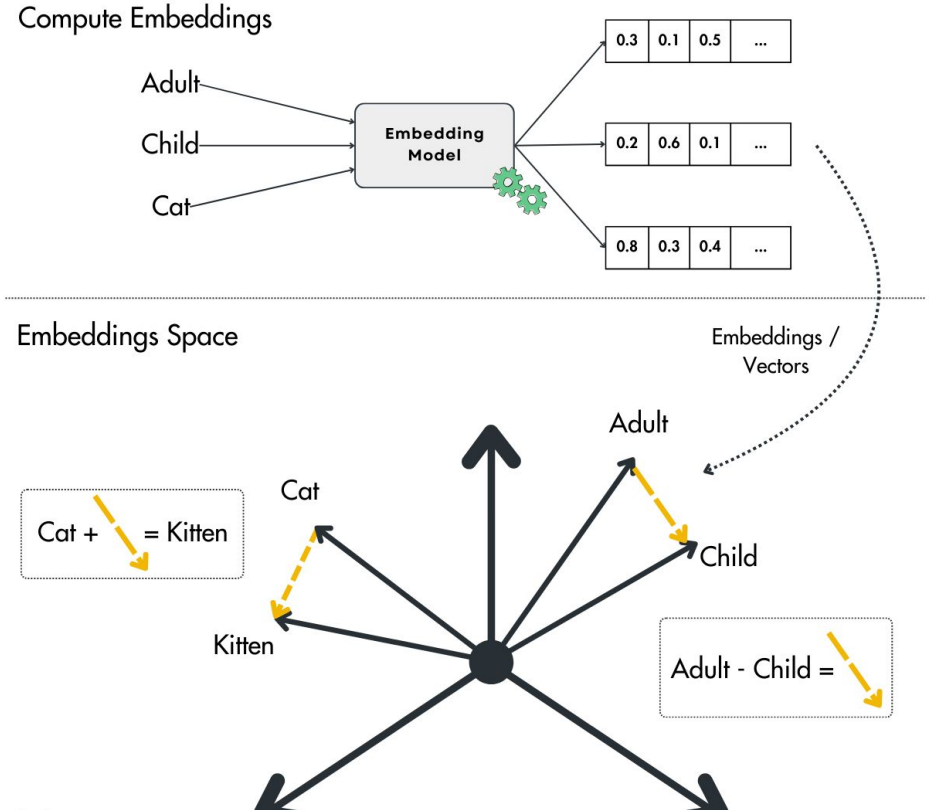


Embeddings: משמעות כמיקום במרחב.

- **טוקן** ממופה **לוקטור**
- **וקטור** הוא **נקודה במרחב רב-ממדי**.
- **הקשרים דומים** מקבלים מיקומים **קרובים**.

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**



Embeddings: משמעות כמיקום במרחב.

"השערת ההתפלגות" (**distributional hypothesis**): קובעת כי מילים המופיעות בהקשרים (Contexts) דומים נוטות לשאת משמעות דומה.

dog ו-***cat*** מופיעות שתיהן ליד ***pet***, ***food***, ***vet***, ולכן הייצוגים הווקטוריים שלהן (Embeddings) ימוקמו קרוב זה לזה במרחב הסמנטי.

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

Embeddings: קרבה בין וקטורים במרחב.

שאלה: איך מודדים "קרבה"?

ד״ר ולד לנדא

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

ניסוח פורמלי - embedding ודמיון קוסינוס:

פונקציית ה-embedding E ממפה כל טוקן t מאוצר המילים $\mathcal{V}$ לווקטור של d (מימדים) מספרים ממשיים:

$$E : \mathcal{V} \to \mathbb{R}^d, \qquad \mathbf{e}_t = E(t) = (e_{t,1},\ e_{t,2},\ \ \dots,\ e_{t,d})$$

$\mathbb{R}^d$ היא קבוצת כל הווקטורים של d מספרים ממשיים; $e_{t,k}$ הוא הרכיב ה-k של הווקטור $\mathbf{e}_t$. במודלים גדולים d הוא בסדרי גודל של אלפים. הערכים של E הם חלק מהמשקלים שנלמדים באימון.

דמיון קוסינוס לכל שני וקטורים $\mathbf{a} = (a_1, \dots, a_d)$ ו-$\mathbf{b} = (b_1, \dots, b_d)$ - למשל שני embeddings, $\mathbf{a} = \mathbf{e}_t$ ו- $\mathbf{b} = \mathbf{e}_{t'}$ של שני טוקנים t, t' - דמיון הקוסינוס הוא הקוסינוס של הזווית ביניהם:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|}$$

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

Embeddings: קרבה בין וקטורים במרחב.

שאלה: איך מודדים "קרבה"?

המדד הנפוץ הוא **דמיון קוסינוס** (**cosine similarity**): מודדים את הזווית בין שני וקטורים.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$$

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Embeddings: קרבה בין וקטורים במרחב.

דוגמה:

כלב = [0.10 ,0.80 ,0.90]

חתול = [0.20 ,0.75 ,0.85]

מכונית = [0.95 ,0.20 ,0.10]

מה הדימיון?

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

Embeddings: קרבה בין וקטורים במרחב.

החישוב המלא. כאן $d = 3$, ונסמן $\mathbf{b} = \mathbf{e}_{\text{לוחם}} = (0.85, 0.75, 0.20)$ -ו $\mathbf{a} = \mathbf{e}_{\text{בלכ}} = (0.90, 0.80, 0.10)$:

$$\mathbf{a} \cdot \mathbf{b} = 0.90 \cdot 0.85 + 0.80 \cdot 0.75 + 0.10 \cdot 0.20 = 0.765 + 0.600 + 0.020 = 1.385$$

$$\|\mathbf{a}\| = \sqrt{0.90^2 + 0.80^2 + 0.10^2} = \sqrt{1.46} \approx 1.208, \qquad \|\mathbf{b}\| = \sqrt{0.85^2 + 0.75^2 + 0.20^2} = \sqrt{1.325} \approx 1.151$$

$$\cos(\mathbf{e}_{\text{בלכ}}, \mathbf{e}_{\text{לוחם}}) = \frac{1.385}{1.208 \times 1.151} \approx 0.996$$

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$$

Embeddings: קרבה בין וקטורים במרחב.

דוגמה:

כלב = [0.10 ,0.80 ,0.90]

חתול = [0.20 ,0.75 ,0.85]

מכונית = [0.95 ,0.20 ,0.10]

מה הדימיון?

דמיון(כלב, חתול) = 0.996 ← כמעט אותו כיוון

דמיון(כלב, מכונית) = 0.293 ← כיוונים שונים

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

Embeddings: מה קורה כאשר מדובר בקוד?

אותו עיקרון עובד על קוד. במרחב של המודל:

- ***sum*** ו-***total*** קרובים;
- ***list*** ו-***array*** קרובים;
- ***for i in range(len(items))*** ו-***for item in items*** מייצגים רעיון דומה.

זה מה שמאפשר למודל לכתוב קוד לבעיה שמעולם לא ראה במדויק: הוא "**מזהה**" שהבעיה החדשה קרובה לבעיות שראה, ומרכיב פתרון מתוך דפוסים קרובים.

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

Embeddings: מה קורה כאשר מדובר בקוד?

החיסרון:

- "**קרוב**" אינו "**זהה**". `to_excel` (קיימת ב-pandas) ו-`to_excel_fast` (לא קיימת) קרובות מאוד במרחב.
- שתיהן נשמעות כמו "כתיבה לאקסל". כשהמודל מחפש את ההמשך הסביר, שכן קרוב ושגוי יכול לנצח.
- זו החוליה הראשונה בהסבר המנגנוני ל**הזיות**.

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

Embeddings: מה קורה כאשר מדובר בקוד?

טעות נפוצה לחשוב:

- "המודל יודע ש-`to_excel_fast` לא קיימת, הוא פשוט טעה."
- המודל לא מחזיק רשימה של "פונקציות שקיימות". יש לו מרחב של קרבות סטטיסטיות - ובמרחב הזה, שם שנשמע נכון נראה כמעט כמו שם נכון.

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

Embeddings: מה קורה כאשר מדובר בקוד?

עקרון מעשי:

- כל שם שנראה **"כמעט מוכר" - פונקציה, פרמטר, ספרייה** - הוא בדיוק המקום **לבדוק בתיעוד**. דווקא השמות הסבירים הם המסוכנים, כי הם לא מדליקים נורה אדומה.

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

Embeddings: מה קורה כאשר מדובר בקוד?

שאלת בדיקה?:

`למה הזיה של requests.fetch מסוכנת יותר מהזיה של requests.banana?`

Vibe Coding

שיעור 2 - חלק ב׳ : **Embeddings**

Embeddings: מה קורה כאשר מדובר בקוד?

שאלת בדיקה?:

למה הזיה של `requests.fetch` מסוכנת יותר מהזיה של `requests.banana`?

תשובה: *כי `fetch` קרוב במשמעות ל-`get` (שקיים), ולכן נראה סביר לחלוטין ולא מעורר חשד. `banana` הייתה נתפסת מיד.*

Vibe Coding

שיעור 2 - חלק ג׳: **(Attention) Transformer**

חשיבות:

Embedding - נקבע לכל טוקן **בנפרד**. ויש צורך שהמשמעות במשפט תהיה קשורה להקשר (context).

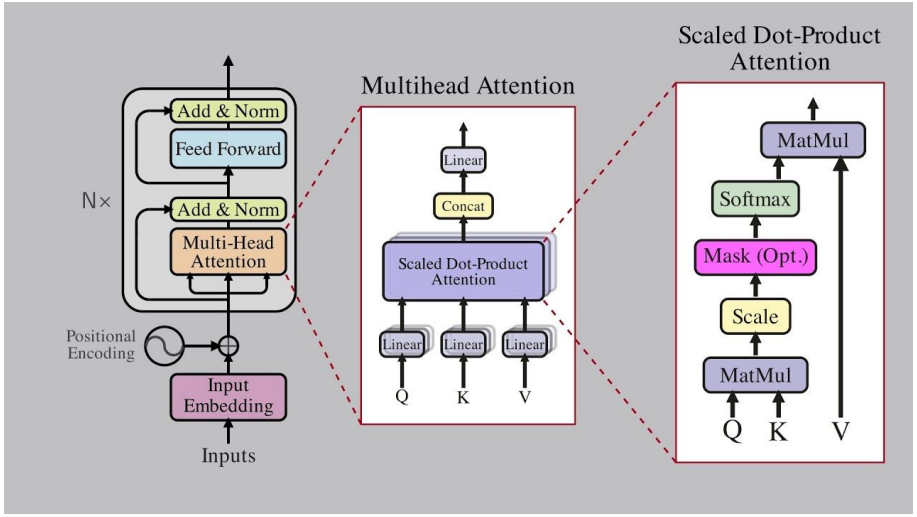
- במשפט "`x = x + 1`", ה-`x` הראשון הוא יעד ההשמה וה-`x` השני הוא הערך הקודם.
- Attention הוא המנגנון שמעדכן את הייצוג של כל טוקן לפי הטוקנים שסביבו.

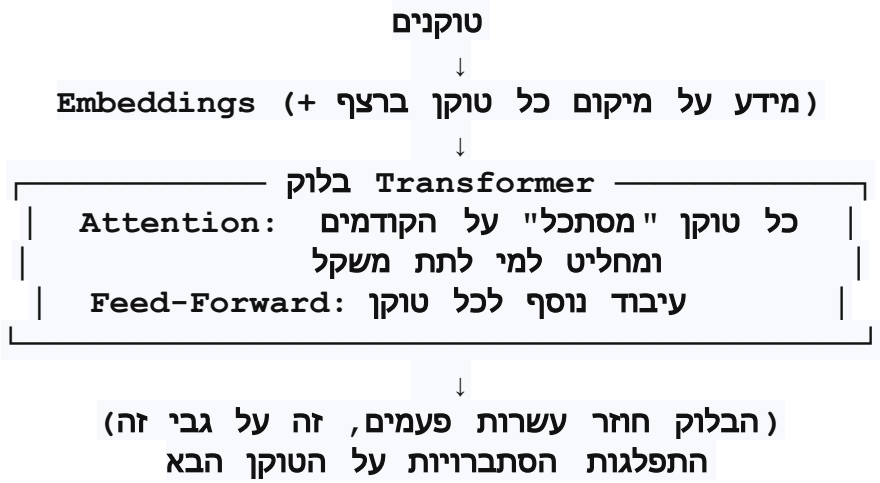
Vibe Coding

שיעור 2 - חלק ג׳: **(Attention) Transformer**

הסבר:

ארכיטקטורת **Transformer** הוצגה ב-2017 במאמר "Attention Is All You Need" והיא הבסיס לכל מודלי השפה הגדולים של היום.

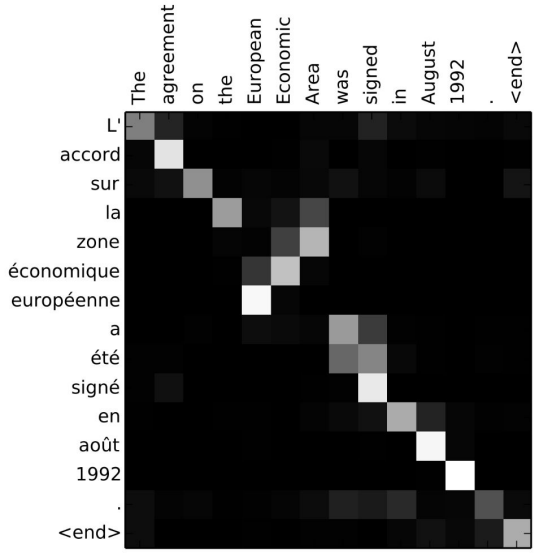




Vibe Coding

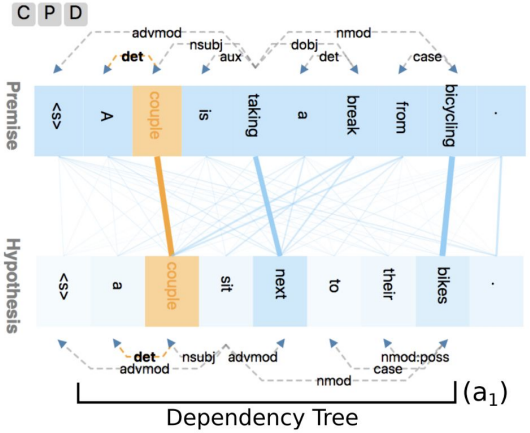
שיעור 2 - חלק ג׳: **(Attention) Transformer**

Attention: לכל טוקן, המודל מחשב "ציון רלוונטיות" לכל אחד מהטוקנים שלפניו, והופך את הציונים למשקלים שסכומם 1. הייצוג החדש של הטוקן הוא ממוצע משוקלל של המידע מכל הטוקנים, לפי המשקלים האלה.



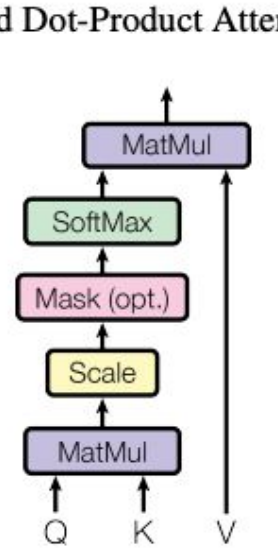
Attention-matrix heatmap

Bahdanau, et al. 2015. Neural machine translation by jointly learning to align and translate. In Proc. ICLR.



Bi-partite graph representation

Shusen Liu, et al. 2018. Visual interrogation of attention-based models for natural language inference and machine comprehension. In EMNLP: System Demonstrations.



Vibe Coding

שיעור 2 - חלק ג': **(Attention) Transformer**

ניסוח פורמלי - Attention:

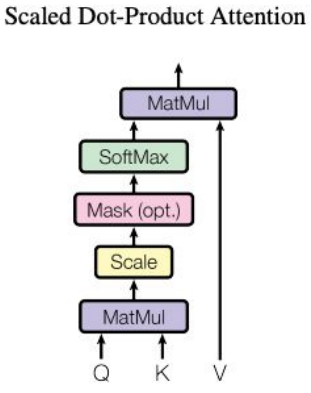
הקלט ל-attention: הרצף מכיל n טוקנים, $t_1, \dots, t_n$. לכל מקום i יש וקטור **ייצוג** $\mathbf{h}_i \in \mathbb{R}^d$

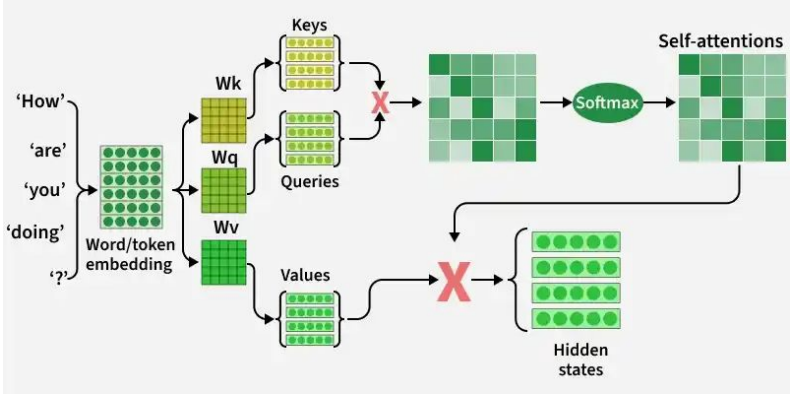
- $\mathbf{h}_i = \mathbf{e}_i + \mathbf{r}_i$, כאשר $\mathbf{e}_i = E(t_i)$ הוא ה-embedding של הטוקן שבמקום i, ו-$\mathbf{r}_i \in \mathbb{R}^d$ הוא וקטור שמקודד את **המיקום** ברצף ("מידע על מיקום כל טוקן" בתרשים למעלה). בלי $\mathbf{r}_i$, המודל לא היה יודע מה הסדר של הטוקנים.

מכל ייצוג $\mathbf{h}_j$, בכל מקום $j = 1, \dots, n$, המודל מחשב שלושה וקטורים בעזרת שלוש מטריצות משקלים שנלמדות באימון - W_Q, W_K, W_V:

- **Query** - "מה אני מחפש":
- **Key** - "מה אני מציע":
- **Value** - "התוכן שלי":

$$\mathbf{q}_j = W_Q \mathbf{h}_j$$
$$\mathbf{k}_j = W_K \mathbf{h}_j$$
$$\mathbf{v}_j = W_V \mathbf{h}_j$$





Vibe Coding

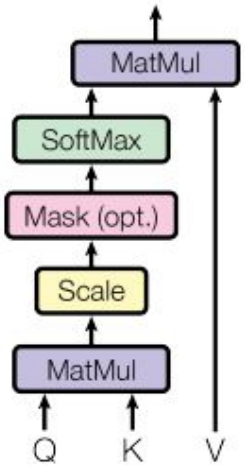
שיעור 2 - חלק ג׳: **(Attention) Transformer**

ניסוח פורמלי - Attention:

כעת נתמקד במקום **אחד**, i - הטוקן שאת הייצוג שלו מעדכנים. הוא משתמש ב-query **שלו**, $\mathbf{q}_i$, וב-keys וב-values של כל המקומות $j \leq i$ - כלומר של עצמו ושל כל הטוקנים שלפניו.

1. **ציון רלוונטיות** של המקום j עבור המקום i - מכפלה סקלרית: $s_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$

Scaled Dot-Product Attention



2. **משקלים** - הציונים הופכים למשקלים חיוביים שסכומם 1, בעזרת **softmax**:

$$\alpha_{ij} = \frac{e^{s_{ij}}}{\sum_{j'=1}^{i} e^{s_{ij'}}} \quad \text{for } j \leq i, \qquad \sum_{j=1}^{i} \alpha_{ij} = 1$$

3. **הפלט** - ממוצע משוקלל של ה-values:

$$\mathbf{o}_i = \sum_{j=1}^{i} \alpha_{ij}\, \mathbf{v}_j \ \in \mathbb{R}^{d_v}$$

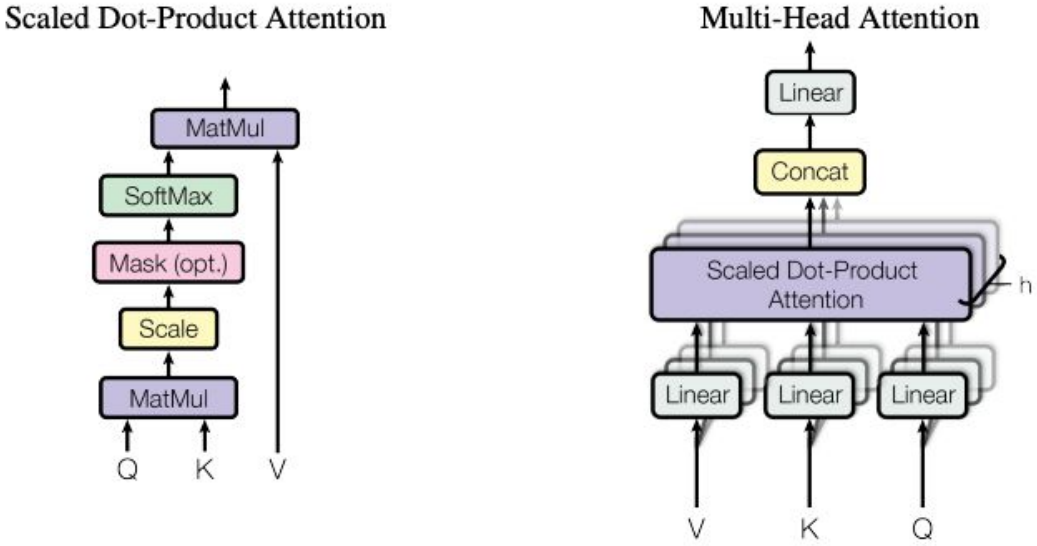
אותו חישוב לכל המקומות בכתיב מטריציוני:

$$\text{Attention}(Q, K, \mathbf{V}) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)\mathbf{V}$$

Vibe Coding

שיעור 2 - חלק ג׳: **(Attention) Transformer**

Multi-head: המודל מריץ כמה מנגנוני קשב במקביל ("ראשים"), וכל אחד לומד לשים לב לסוג אחר של קשר - אחד לתחביר, אחד לשמות משתנים, אחד להתאמת סוגריים, וכן הלאה. (החלוקה הזו נלמדת, ואיננה מוגדרת מראש.)



Vibe Coding

שיעור 2 - חלק ג׳: **(Attention) Transformer**

מודלים שמייצרים טקסט, כל טוקן יכול להסתכל **רק אחורה** - על טוקנים שכבר נכתבו - ולא קדימה. המודל לא יודע מה הוא "יכתוב אחר כך".

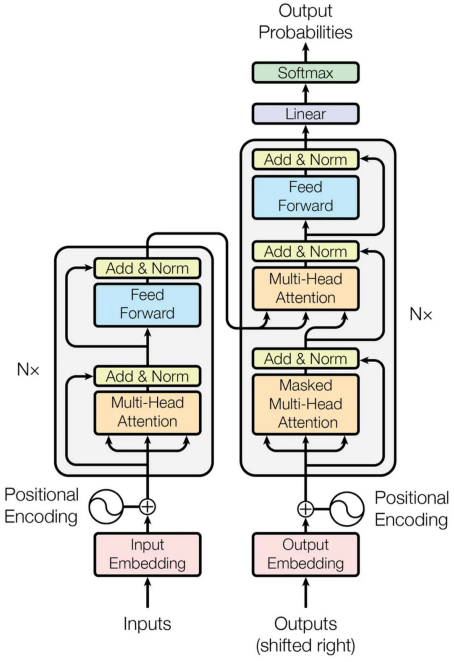
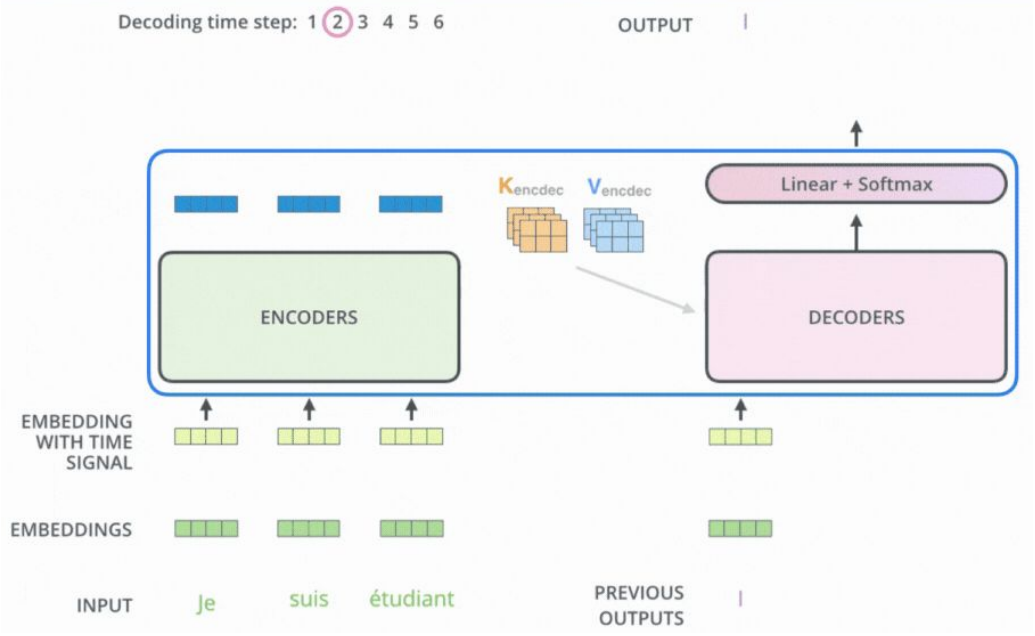


Figure 1: The Transformer - model architecture.



Vibe Coding

שיעור 2 - חלק ג׳: **(Attention) Transformer**

דוגמה: המודל משלים שורת קוד. עד כה נכתב:

```
def total_price(items):
    total = 0
    for item in items:
        total += item.____
```

Vibe Coding

שיעור 2 - חלק ג׳: **(Attention) Transformer**

דוגמה: המודל משלים שורת קוד. עד כה נכתב:

```
def total_price(items):
    total = 0
    for item in items:
        total += item.____
```

מה המילה הבאה? כדי להחליט, המודל "מסתכל אחורה":

למה	משקל קשב (המחשה)	טוקן קודם
שם הפונקציה מרמז שמדובר ב**מחיר**	גבוה	`total_price`
אנחנו ניגשים לשדה של האובייקט הזה	גבוה	`item`
מצביע שצריך ערך מספרי	בינוני	`=+ total`
לא תורם להחלטה	נמוך	`def`

התוצאה הסבירה: `price`.

Vibe Coding

שיעור 2 - חלק ג׳: **(Attention) Transformer**

טעות נפוצה:

"Attention אומר שהמודל מבין את הקוד כמו מתכנת." Attention הוא מנגנון לשקלול הקשר - חזק מאוד, אבל סטטיסטי.

Vibe Coding

שיעור 2 - חלק ג׳: **(Attention) Transformer**

טעות נפוצה:

"Attention אומר שהמודל מבין את הקוד כמו מתכנת." Attention הוא מנגנון לשקלול הקשר - חזק מאוד, אבל סטטיסטי.

עקרון מעשי:

- **שמות ברורים ומדויקים הם הקשר שהמודל משתמש בו בפועל.** פונקציה בשם `process_data` נותנת למודל מעט מאוד לעבוד איתו;
- `calculate_monthly_vat(invoices)` נותנת הרבה.

Vibe Coding

שיעור 2 - חלק ד׳ :**ניבוי הטוקן, וטמפרטורה**

הסבר: לולאת ניבוי (**autoregressive generation**)
בכל צעד:

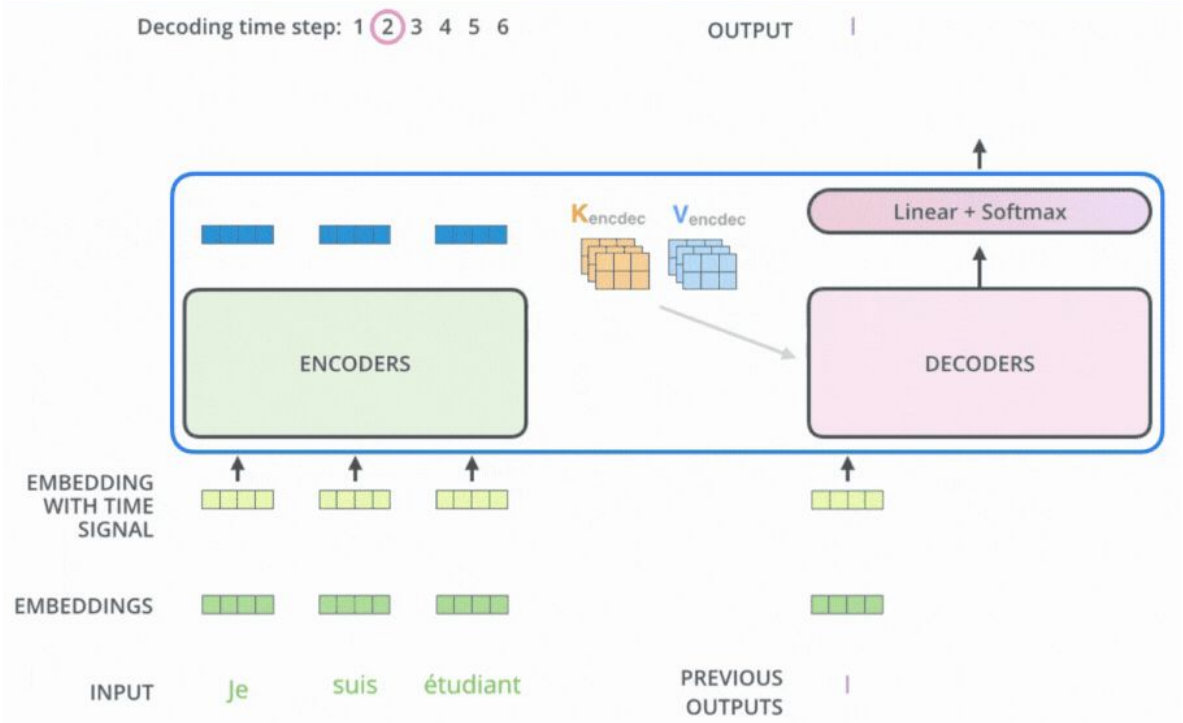
1. המודל מקבל את כל הטוקנים עד כה (הקשר).
2. בסוף ה-Transformer מתקבל **ציון גולמי** (logit) לכל טוקן אפשרי באוצר המילים.
3. הציונים הופכים להתפלגות הסתברויות באמצעות פונקציית **softmax**.
4. **נבחר** טוקן אחד מתוך ההתפלגות.
5. הטוקן מתווסף לרצף - וחוזרים לשלב 1.

לולאת **ייצור אוטו-רגרסיבי (autoregressive generation)**: כל פלט הופך לחלק מהקלט של הצעד הבא. המודל לא מתכנן את הפונקציה מראש ואז **"מקליד" אותה** - הוא **בונה אותה צעד-צעד**, וכל צעד מחייב את כל הצעדים שאחריו.

Vibe Coding

שיעור 2 - חלק ד׳ :**ניבוי הטוקן, וטמפרטורה**

הסבר: לולאת ניבוי (**autoregressive generation**)



Vibe Coding

שיעור 2 - חלק ד׳ :**ניבוי הטוקן, וטמפרטורה**

ניסוח פורמלי - ייצור אוטו-רגרסיבי:

x הוא הקלט - **הפרומפט** שלכם (כרצף טוקנים). $y = (t_1, t_2, \dots, t_m)$ היא **התשובה** שהמודל מייצר: כאן הטוקנים $t_1, \dots, t_m$ ממוספרים **מתחילת התשובה**, ו-m הוא אורך התשובה. $t_{<i}$ הוא קיצור ל"כל טוקני התשובה שלפני המקום i", כלומר $t_1, \dots, t_{i-1}$ (ב-$i = 1$ זהו רצף ריק). θ הם כל המשקלים של המודל.

בצעד i המודל מעבד את x ואת $t_{<i}$ ומחשב וקטור ציונים $\mathbf{z}^{(i)} \in \mathbb{R}^{|\mathcal{V}|}$ - ציון גולמי (logit) אחד לכל טוקן t באוצר המילים. נסמן ב- $z_t^{(i)}$ את הציון של הטוקן. ה-**softmax** הופך את הציונים להתפלגות:

$$p^{(i)}(t) = P_\theta\left(t_i = t \mid x,\ t_{<i}\right) \qquad p^{(i)}(t) = \frac{e^{z_t^{(i)}/T}}{\sum_{t' \in \mathcal{V}} e^{z_{t'}^{(i)}/T}}$$

כלומר $p^{(i)}(t)$ היא ההסתברות שהטוקן שייבחר במקום i יהיה t, בהינתן הפרומפט וכל מה שכבר נכתב.

תשובה שלמה: לפי כלל השרשרת בהסתברות, ההסתברות של התשובה כולה היא מכפלה של כל הצעדים:

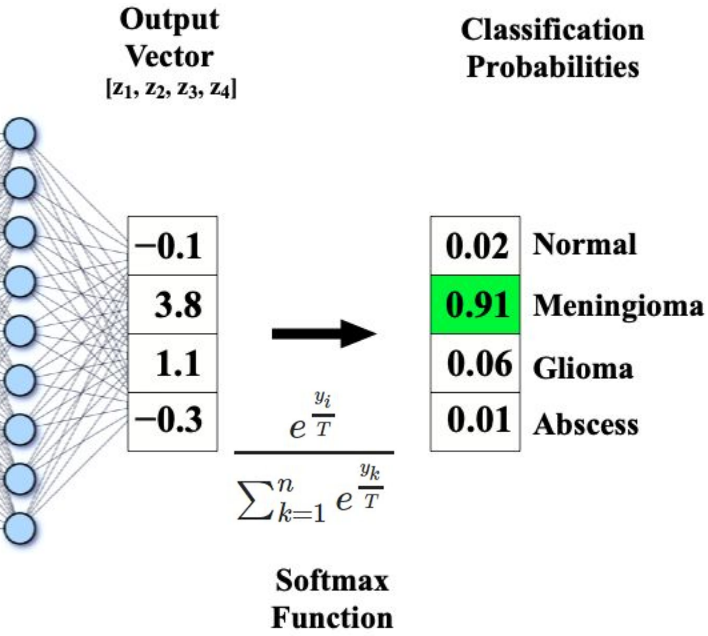
$$P_\theta(y \mid x) = \prod_{i=1}^{m} P_\theta(t_i \mid x,\ t_{<i})$$

Vibe Coding

שיעור 2 - חלק ד׳ :**ניבוי הטוקן, וטמפרטורה**

טמפרטורה:

לפני ה-**softmax**, מחלקים את כל הציונים בפרמטר **T** (טמפרטורה):



- T **קטן (למשל 0.2)**: ההבדלים בין הציונים מתעצמים ← ההתפלגות "חדה" ← כמעט תמיד נבחר הטוקן המוביל.
- **T = 1**: ההתפלגות כפי שהמודל חישב אותה.
- T **גדול (למשל 1.5)**: ההבדלים מתכווצים ← ההתפלגות "שטוחה" ← אפשרויות פחות סבירות נבחרות לעיתים קרובות יותר.
- **T = 0**: תמיד נבחר הטוקן המוביל (greedy). (בפועל, גם ב-T=0 ייתכנו הבדלים קטנים בין הרצות, מסיבות טכניות.)

Vibe Coding

שיעור 2 - חלק ד׳ :**ניבוי הטוקן, וטמפרטורה**

טמפרטורה - דוגמה : המודל כתב `for i in` וצריך להמשיך.

מועמד	ציון גולמי	T = 0.2	T = 0.7	T = 1.0	T = 1.5
range(len(x)):	4.0	**99.80%**	82.10%	68.30%	53.00%
numbers:	2.7	0.20%	12.80%	18.60%	22.30%
data.items():	1.7	0.00%	3.10%	6.80%	11.40%
enumerate(items):	1.2	0.00%	1.50%	4.20%	8.20%
zip(a, b):	0.5	0.00%	0.60%	2.10%	5.10%

- **טמפרטורה גבוהה מגדילה את הסיכוי לסטייה.**

Vibe Coding

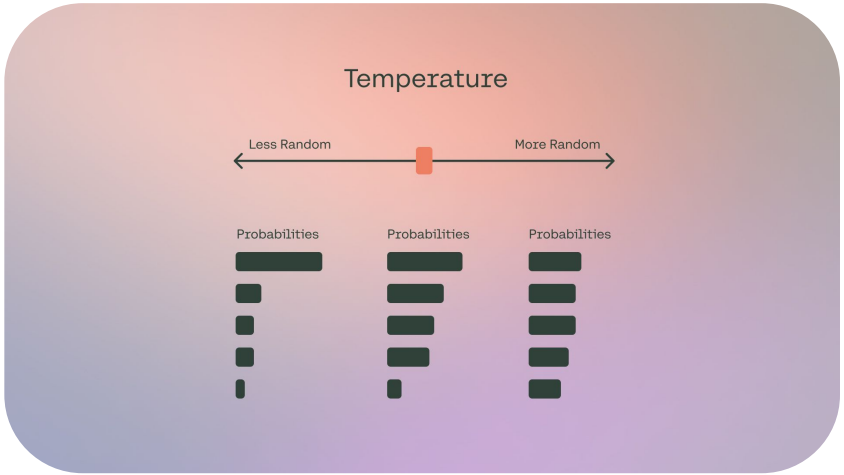
שיעור 2 - חלק ד׳ :**ניבוי הטוקן, וטמפרטורה**

טעות נפוצה:

"טמפרטורה 0 = המודל צודק." **לא**. **טמפרטורה 0 = המודל עקבי**: הוא בוחר תמיד באפשרות שהוא עצמו מעריך כסבירה ביותר. אם ההערכה שלו שגויה, הוא יטעה - באותו אופן בדיוק, בכל פעם.

עקרונות:

- **לקוד - טמפרטורה נמוכה, כשיש שליטה.**
- **לסיעור מוחות - טמפרטורה בינונית-גבוהה.**
- **בלי שליטה בטמפרטורה: הריצו את אותה בקשה פעמיים-שלוש -** תשובות זהות - סימן שההתפלגות חדה.



Vibe Coding

שיעור 2 - חלק ה׳ :**חלון ההקשר (Context Window)**

חשיבות:

למודל **אין זיכרון בין שיחות**, **ואין** לו **זיכרון** "רגיל" **בתוך שיחה**. כל מה שהוא **"יודע"** על השיחה הנוכחית הוא מה ש**נמצא** ברגע זה **בחלון ההקשר**. כמעט כל תקלה בשיחות ארוכות ("הוא שכח מה ביקשתי!") נובעת מכאן.



Vibe Coding

שיעור 2 - חלק ה׳ :**חלון ההקשר (Context Window)**

הסבר:

בכל פעם שאתם שולחים הודעה, הכלי **שולח** למודל מחדש **את כל השיחה** (או את מה שנכנס ממנה). המודל לא "זוכר" את ההודעה הקודמת - הוא קורא אותה שוב.

גודל החלון משתנה מאוד בין מודלים - מעשרות אלפי טוקנים ועד כמיליון בחלק מהמודלים. המספר המדויק משתנה כל הזמן. מה שכן צריך להבין: **החלון תמיד סופי**, ובשיחות עבודה אמיתיות על קוד - מגיעים לקצה שלו.

Vibe Coding

שיעור 2 - חלק ה׳ :**חלון ההקשר (Context Window)**

מה קורה כשהחלון מתמלא?:

1. **חיתוך:** כלים רבים משמיטים או מתמצתים (**compacting**) את החלק הישן של השיחה כדי לפנות מקום. הכלל ש"הגדרתם בהודעה הראשונה" (למשל: "רק Python 3.11, בלי ספריות חיצוניות") פשוט איננו יותר בחלון.
2. **עיוורון לקוד שמחוץ לחלון:** אם המודל רואה רק חלק מהפרויקט, הוא עלול לכתוב פונקציה שכבר קיימת, או לקרוא לפונקציה בחתימה שגויה.
3. **ירידה בחדות גם לפני שהחלון מלא:** מחקרים הראו שמודלים מנצלים פחות טוב מידע שנמצא באמצע הקשר ארוך, בהשוואה למידע בתחילתו ובסופו (תופעה שתועדה במאמר "Lost in the Middle", Liu ועמיתים, 2023). כלומר "זה נמצא איפשהו בשיחה" איננו ערובה שהמודל ישתמש בזה.

Vibe Coding

שיעור 2 - חלק ה׳ :**חלון ההקשר (Context Window)**

דוגמה:

```
הודעה 1 (אתם): "כל הקוד בפרויקט הזה – בלי ספריות
חיצוניות, רק standard library."
... 40 הודעות, הרבה קוד מדבק ...
הודעה 42 (אתם): "תוסיף קריאה של קובץ CSV."
תשובת המודל: import pandas as pd
"df = pd.read_csv("data.csv)
```

- המודל לא "התעלם" מכם. סביר שההוראה מהודעה 1 כבר לא הייתה בחלון, או שהייתה קבורה באמצע הקשר ארוך.

Vibe Coding

שיעור 2 - חלק ה׳ :**חלון ההקשר (Context Window)**

טעות נפוצה:

"אמרתי לו את זה כבר, **הוא אמור לזכור.**" המודל לא זוכר - הוא קורא מחדש את מה שנשאר בחלון.

עקרונות מעשיים:

1. **שיחה חדשה לכל משימה חדשה**. שיחה ארוכה אחת לכל הפרויקט היא מתכון לחלון עמוס.
2. **הוראות חשובות - חזרו עליהן** בבקשה הרלוונטית, או שמרו אותן בקובץ הנחיה קבוע.
3. **הדביקו רק את מה שרלוונטי** - לא את כל הפרויקט "ליתר ביטחון".
4. כשמשהו מתחיל להשתבש בשיחה ארוכה - פתחו שיחה חדשה עם סיכום קצר של מה שחשוב.

Vibe Coding

שיעור 2 - חלק ו׳ :**למה קורות הזיות?**

ניסוח פורמלי - תקציב החלון:

חלון ההקשר מכיל לכל היותר L טוקנים. כל אחד מהרכיבים שציינו "עולה" מספר טוקנים: n_{system} - הוראות המערכת של הכלי; n_{history} - **כל** ההודעות הקודמות בשיחה, שלכם ושל המודל, יחד; n_{files} - קבצים וקוד שהודבקו; n_{prompt} - ההודעה הנוכחית; n_{answer} התשובה שהמודל כותב. כל בקשה חייבת לקיים:

$$n_{\text{system}} + n_{\text{history}} + n_{\text{files}} + n_{\text{prompt}} + n_{\text{answer}} \leq L$$

Vibe Coding

שיעור 2 - חלק ו׳ :**למה קורות הזיות?**

***__הזיה איננה תקלה נדירה__ במנגנון - __היא תוצר לוואי צפוי__ של מנגנון שמייצר טקסט סביר בלי שלב אימות. ולכן: **האימות הוא התפקיד שלכם.**



Vibe Coding

שיעור 2 - חלק ו׳ :**למה קורות הזיות?**

כמה זה נפוץ? נתון מחקרי:

במחקר "We Have a Package for You!" (Spracklen ועמיתים, USENIX Security 2025), החוקרים ייצרו 576,000 דוגמאות קוד ב-Python וב-JavaScript באמצעות 16 מודלים, ובדקו כל חבילה שהקוד המליץ עליה מול מאגרי PyPI ו-npm.

- במודלים מסחריים, בממוצע לפחות **5.2%** מהחבילות שהומלצו לא היו קיימות; במודלי קוד פתוח - **21.7%**.
- נמצאו יותר מ-**205,000** שמות חבילות בדויים ייחודיים.
- הזיות רבות **חזרו על עצמן** בהרצות חוזרות - כלומר, אותו שם בדוי מופיע שוב ושוב.

Vibe Coding

שיעור 2 - חלק ו׳ :**למה קורות הזיות?**

ניסוח פורמלי - מה המשמעות של **5.2%** לפרויקט:
נסמן ב-p_{hal} את ההסתברות שהמלצה **בודדת** על חבילה היא חבילה בדויה (במחקר: $p_{\text{hal}} = 0.052$ במודלים מסחריים), וב-R את מספר החבילות שהמודל המליץ עליהן בפרויקט. אם ההמלצות בלתי-תלויות (הנחה מפשטת), ההסתברות שכל R ההמלצות אמיתיות היא $(1 - p_{\text{hal}})^R$, ולכן ההסתברות שתהיה **לפחות** חבילה בדויה אחת היא המשלים:

$$P(\text{at least one wrong package}) = 1 - (1 - p_{\text{hal}})^R$$

תרגיל: תחשבו עבור R = 1, 5 ,10.

Vibe Coding

שיעור 2 - חלק ו׳ :**למה קורות הזיות?**

ניסוח פורמלי - מה המשמעות של **5.2%** לפרויקט:
נסמן ב-p_{hal} את ההסתברות שהמלצה **בודדת** על חבילה היא חבילה בדויה (במחקר: $p_{\text{hal}} = 0.052$ במודלים מסחריים), וב-R את מספר החבילות שהמודל המליץ עליהן בפרויקט. אם ההמלצות בלתי-תלויות (הנחה מפשטת), ההסתברות שכל R ההמלצות אמיתיות היא $(1 - p_{\text{hal}})^R$, ולכן ההסתברות שתהיה **לפחות** חבילה בדויה אחת היא המשלים:

$$P(\text{at least one wrong package}) = 1 - (1 - p_{\text{hal}})^R$$

תרגיל: תחשבו עבור R = 1, 5 ,10.

שיעור "קטן" של 5.2% להמלצה בודדת הופך לסיכוי של כמעט 1 ל-4 בפרויקט עם 5 תלויות שהומלצו על ידי AI. לכן הבדיקה ב-PyPI היא לא "זהירות יתר" אלא הכרח.

ד"ר ולד לנדא

Vibe Coding

שיעור 2 - חלק ו': **למה קורות הזיות?**

טקסונומיה: חמישה סוגי הזיות בקוד, ואיך מאמתים כל אחד:

סוג	דוגמה	אימות
1. פונקציה/מתודה - שם שלא קיים	`df.to_excel_fast("out.xlsx")` ← `AttributeError`	הרצה (השגיאה מיידית) + התיעוד
2. פרמטר - הפונקציה קיימת, הפרמטר לא	`requests.get(url, retries=3)` ← `TypeError`	הרצה + חתימת הפונקציה בתיעוד. ערמומית: חצי מהשורה נכון
3. חבילה - ספרייה שלא קיימת	`pip install fast-excel-writer-pro` (שם בדוי)	לפני ההתקנה: pypi.org — מתי פורסמה, כמה הורדות, ריפו פעיל
4. גרסה - קיימת, אבל לא בגרסה שלכם	`case/match` על Python 3.8 (נוסף ב-3.10)	"Added in version" + `python --version` בתיעוד
5. התנהגות - הקוד רץ, ההסבר שגוי	"מטפלת ברשימה ריקה", אבל `average([])` ← `ZeroDivisionError`	רק הרצה עם קלט קצה. המסוכנת ביותר — הכול "נראה" נכון